

What ten other projects already know

A plain-language summary of the prior-art review (issue #175)

6 August 2026 · ten repositories read at pinned commits · nine test suites actually run

The one-paragraph version

We read the ten public projects that are built most like mcgyvr, at exact commits, and *ran their tests* rather than trusting their READMEs. Then six independent readers checked the first pass. The most useful things we found were not clever features — they were **mistakes these projects made and paid for**. Four of them are mistakes mcgyvr is currently positioned to repeat, and **one of them mcgyvr has already made**: our capacity limit doesn't work across processes, and our own workflow runs multiple processes.

The scoreboard

10

repositories
read

9

test suites
actually run

2

issues filed
so far

4

new decisions
for you

Why "actually run" matters. A project's README says what its author hoped. Its tests say what its author was burned by. We only counted a finding if we could point at code at a specific commit, or run something. The first pass said four test suites "couldn't be run here" — **three of those four ran fine on a second attempt**. That is a lesson about us, not about them.

1. First, the map: who sits where

The most important decision in this review was *which* projects to read. It turns out there are two very different kinds of tool, and most famous ones are the wrong kind for us to learn from.

THE CALLER — a human sits here and drives it

The human reading the diff is the safety check. So there is no automatic gate to learn from.

cline

orca

multica

opencode

goose

OpenHands

the task contract — the line mcgyvr sits below

BELOW THE CONTRACT — this is where mcgyvr lives

Work is split up, sent to a model, and judged by a test the model did not write. These have gates.

Agentless

moatless

auto-code-rover

SWE-agent

micro-agent

OpenHands SDK

plandex

RA.Aid

SWE-ReX

oh-my-
openagent

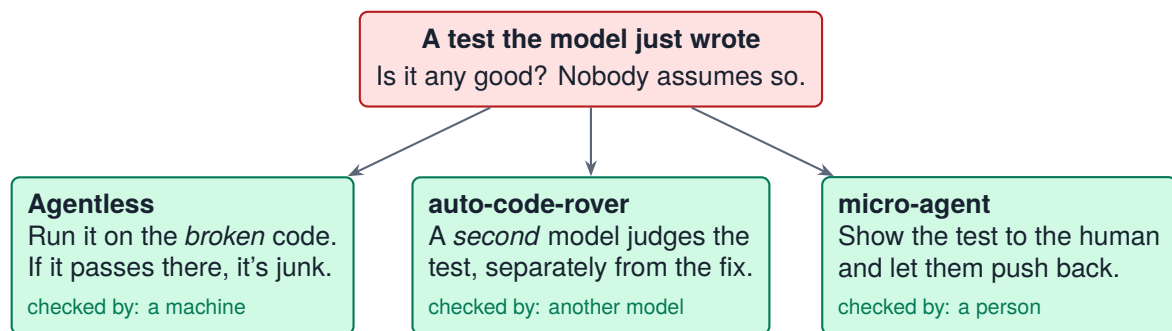
aider

*split its own repo in two,
July 2026 — the caller and
the engine are now
separate projects*

The best thing we found all week. Ten days before we looked, OpenHands — the biggest project of this type, 83,000 stars — **split its repository in half along exactly this line**. One repo is now the caller; the other is the engine underneath. We drew that line from theory. They drew it in production and deleted 486,115 lines doing it. That is independent confirmation that the line is real.

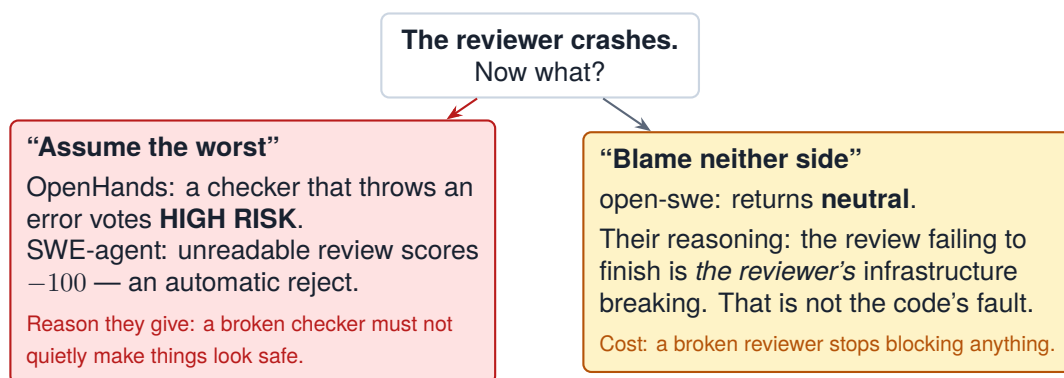
2. Finding 1: a test needs its own check

Every one of these systems asks a model to write a test that proves the bug is fixed. And **not one of them trusts that test**. Each invented a different way to check the checker:



And when the checker itself breaks?

That is the question mcgyvr has open (#179): if the reviewer crashes, is that the *work's* fault or the *reviewer's* fault? Two projects shipped an answer and they agree; one disagrees. This is a real fork in the road, and you have to pick.



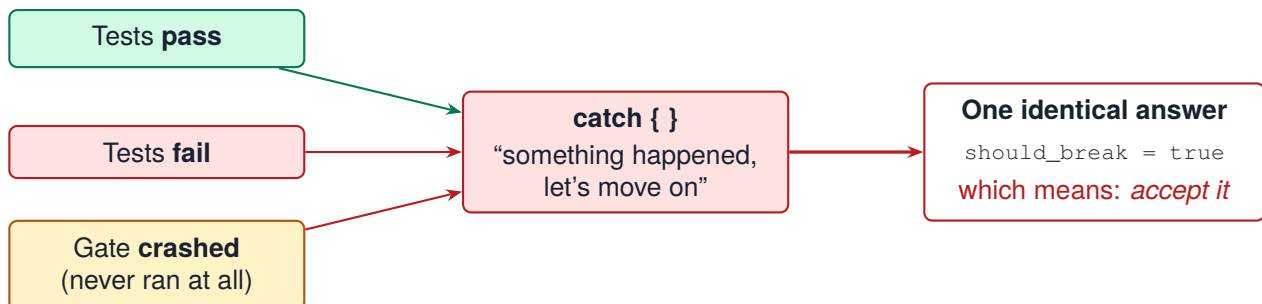
What this means for you

You don't need to invent an answer to #179 — you need to *choose* between two that already exist and have been run in anger. "Fail closed" is safer and noisier. "Neutral" keeps the pipeline moving and lets a broken reviewer wave things through.

3. Finding 2: the scariest thing we found

RA.Aid has a safety gate: you give it a test command, and it runs your tests before accepting the model's work. **That gate has not run since 27 January 2025.** It crashes instantly, every time, on a typo-level bug — it passes an argument the function doesn't accept.

That alone would be bad. Here is what makes it genuinely dangerous:



Three completely different situations — everything worked, everything is broken, the check never happened — come out the other side as **the same single value**. Nothing downstream can tell them apart. We verified this by running their own code three ways.

And this is why nobody noticed

The gate has six tests. All six pass. They pass because each one **replaces the real function with a fake** that politely accepts any argument — including the argument that breaks the real one.

The mock is the thing that kept it green for eighteen months.



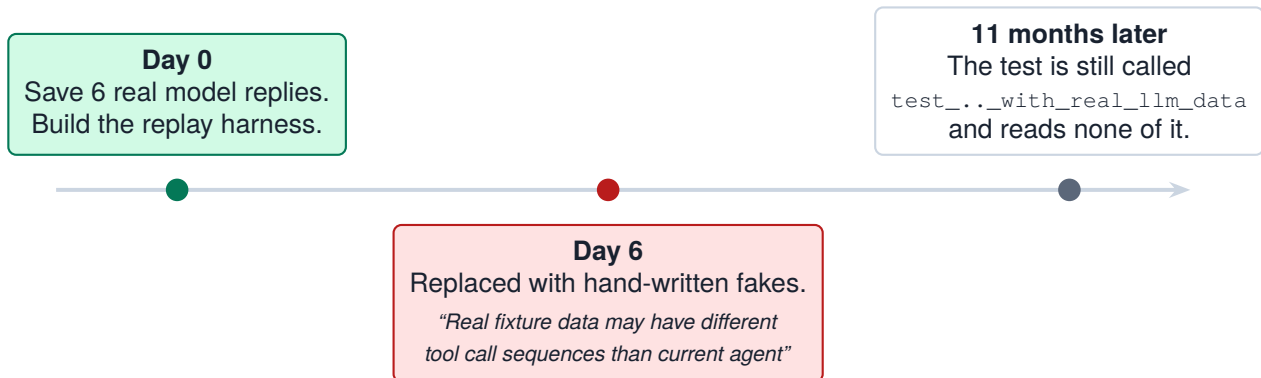
The question this puts to mcgyvr

Two things, both narrow enough to answer today:

1. Should the piece of mcgyvr that *actually shells out to run the tests* be allowed to be faked in *any* test? RA.Aid says no, expensively.
2. Can a yes/no value carry *passed*, *failed* **and** *never ran*? It cannot. Right now we have two of those three sharing a slot.

4. Finding 3: saved model replies rot — unless you save the right part

We have an open idea (#184) to start saving what the models say back to us, since we already have it and throw it away. Good idea, but OpenHands did exactly that and it died within a week. Here is the timeline:



Why it rotted, and what to save instead

Don't save: the whole conversation

A full recording of one run: every step, every tool call, in order.

Breaks the moment you change a tool.
Your own product changing kills the fixture.

Do save: what the parser has to chew

Just the raw reply text and the shape your parser must cope with.

Stays valid for years.
Only breaks if the *model* changes its style.

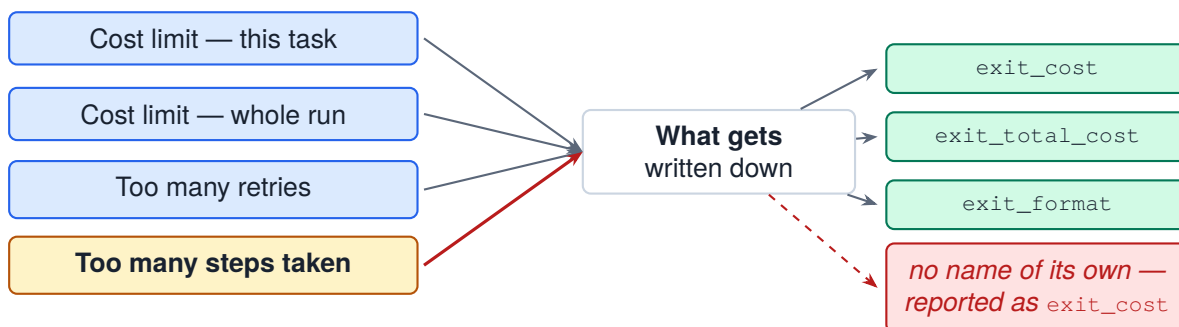
aider is the proof of the good version: **99,961 lines** of real captured replies, checked against a reference file — and crucially it keeps **33 replies that failed to parse**, as treasured examples rather than embarrassments. That corpus cost them one `git add` of a log they already had.

What this means for you

#184 is right, but it should be reworded before anyone builds it: capture **what the parser reads**, not **what the run did**. Same effort, one lasts and one doesn't.

5. Finding 4: limits that lie about why they stopped

SWE-agent looks exemplary here. It has four separate spending limits, each with its own name, so a run can always say why it gave up. Except it can't:



The "too many steps" limit was written as a *kind of* cost limit, purely for convenience in the code. The side effect: **a run that ran out of steps reports that it ran out of money**. Whoever reads that log later will tune the wrong dial.

There's a second one in the same file. A loop says "stop after 3 retries" — but two of the ways it retries **never increment the counter**. So the limit you declared is not the limit that operates; the budget stops it eventually, by accident.

The rule worth stealing: every path that burns an iteration must count one, and every limit you declare needs its own name in the record.

6. Finding 5: everyone picks models by name, nobody by evidence

aider has measured more model behaviour than anyone in this field — years of public benchmark runs. So we expected its model routing to be driven by that data. It isn't. **Nothing reads it.**



How many of the 342 models it routes have a measurement behind them?

17%

83% — chosen by judgement, and never revisited

The example that should matter most to you

Their own published data, for exactly the class of model your rigs run:

Qwen2.5-Coder-32B — same model, two ways of asking it to edit

Format “diff”

Replies we could parse: **71.6%**

148 replies unusable

← this is the one their code picks

Format “whole”

Replies we could parse: **99.6%**

1 reply unusable

← their own data says use this

A 28-point swing in whether the model's answer is even *usable*, from changing nothing but how you ask. The weaker your model, the more this matters — and local models are the whole point of mcgyvr.

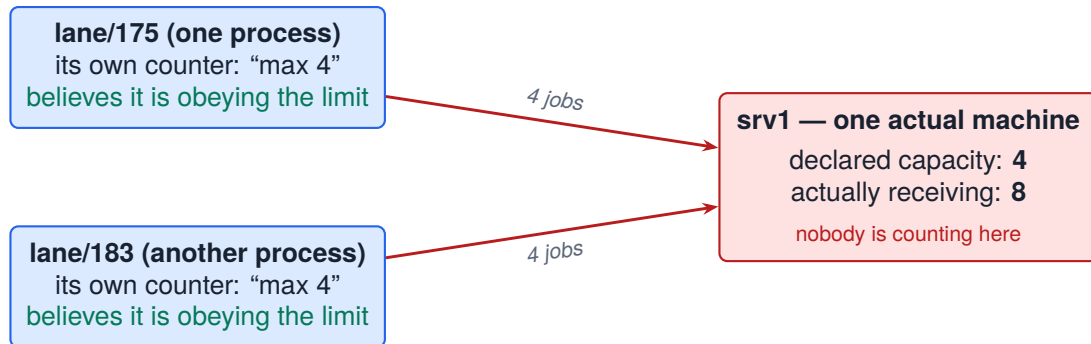
What this means for you (#162, the routing question)

There is a cheap number nobody is using: **what fraction of this worker's replies did we manage to parse?** mcgyvr already produces it on every single dispatch and throws it away. Routing on *measured reply quality* instead of *model name* would be genuinely new — nobody in this survey has built it.

7. Finding 6: the one that's about us

This came out of the concurrency annex, and it is the most actionable thing in the whole review.

mcgyvr limits how many jobs it sends to a machine at once. The limit is a counter held **inside one running process**. But the machines are shared, and your own workflow runs **several processes at the same time** — that's what lanes in separate worktrees are.



Two details that make it worse

- When a job does have to wait for a slot, it waits **forever** — no timeout. Of every design we looked at, ours is the only one that can block indefinitely.
- The file explains four design decisions in careful detail, and calls one scenario "the worst failure this module could have". **The word "process" does not appear anywhere in it.** So this case wasn't decided against — it was never considered, which is why nothing flags it.

All three neighbours handle this better

container-use A lock the <i>operating system</i> holds If the process dies, the OS releases it automatically.	orca Doesn't keep a counter at all — <i>asks</i> how many are running. Survives a restart for free.	multica No coordinator whatsoever. Racers insert a row; the database picks one winner.
---	---	--

multica's version is the elegant one and it is worth a sentence, because **mcgyvr already does exactly this one level up**: claiming a lane is just creating a branch on the server, and two agents racing for the same branch name means exactly one wins. Same trick, already in the building.

8. So what should you actually do?

Four decisions, in the order I'd take them. None is a big build.

1 Fix the capacity limit

It's our bug, it's live, and three worked examples exist. Cheapest fix: a lock the OS releases when the process dies.

2 Ban mocking the test-runner

Decide that the part of mcgyvr that shells out to run tests may never be faked — and give "never ran" its own value, separate from "failed".

3 Reword #184 before building it

Save what the parser reads, not what the run did. Same work; one lasts for years, the other dies in a week.

4 Route on parse-success rate

We already measure it and bin it. Using it would make mcgyvr the first thing here that picks a worker on evidence rather than on its name.

Two things I want to be straight about

Nothing has been filed. The review documents these four; turning them into issues is your call, not mine.

Half the review has not been double-checked. Six independent readers re-checked repositories 6–10. Four of those six found real errors in the first pass — and *every single error pointed the same way*, making the other projects look less capable than they are. Repositories 1–5 have had no second reading. The honest assumption is that they contain the same kind of error, in the same direction.

Where everything lives. Full review: `docs/prior-art-review-2026-08-06.md` on branch `lane/175` — 1,305 lines, every claim carrying the exact commit it came from so anyone who doesn't believe it can go and check. No pull request has been opened, and nothing in it has been reviewed by you yet.